Section 1. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive
Section 2. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 3. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per
Section 4. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier sc
Section 5. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against ac
Section 6. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical h

Section 7. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine prove
Section 8. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 9. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. A
Section 10. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream wi
Section 11. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level fr
Section 12. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dic

Section 13. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pa
Section 14. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 15. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resu
Section 16. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mix
Section 17. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Comp
Section 18. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pay

Section 19. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself,
Section 20. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
Section 21. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger
Section 22. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical
Section 23. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed s
Section 24. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffma

Section 25. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns throu
Section 26. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against a
Section 27. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapti
Section 28. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level fr
Section 29. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocl
Section 30. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,

Section 31. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and codin
Section 32. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level fr
Section 33. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, a
Section 34. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when th
Section 35. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into
Section 36. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse

Section 37. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thre
Section 38. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,
Section 39. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pa
Section 40. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 41. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 42. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream wi

Section 43. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 44. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only w
Section 45. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 46. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File C
Section 47. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engi
Section 48. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hyb

Section 49. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds
Section 50. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pay
Section 51. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-
Section 52. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual cou
Section 53. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision En
Section 54. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing

Section 55. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 56. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tie
Section 57. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionar
Section 58. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into
Section 59. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one c
Section 60. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns throu

Section 61. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when th
Section 62. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-
Section 63. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resu
Section 64. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger
Section 65. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffma
Section 66. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditi

Section 67. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural b
Section 68. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol
Section 69. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patt
Section 70. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one c
Section 71. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger
Section 72. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream wi

Section 73. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds
Section 74. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dic
Section 75. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 76. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
Section 77. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pa
Section 78. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,

Section 79. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pay
Section 80. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 81. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression us
Section 82. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual cou
Section 83. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 84. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural b

Section 85. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural b
Section 86. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier s
Section 87. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocl
Section 88. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, a
Section 89. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine prov
Section 90. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resu

Section 91. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level fr
Section 92. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 93. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 94. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and codin
Section 95. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural block
Section 96. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptiv

Section 97. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 98. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual cou
Section 99. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionar
Section 100. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 101. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hy
Section 102. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr

Section 103. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 104. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com
Section 105. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona
Section 106. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 107. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 108. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in

Section 109. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 110. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 111. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 112. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 113. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 114. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 115. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro
Section 116. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 117. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 118. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbo
Section 119. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 120. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural

Section 121. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 122. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 123. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 124. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti
Section 125. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 126. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,

Section 127. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 128. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 129. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 130. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 131. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffm
Section 132. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti

Section 133. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 134. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 135. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 136. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 137. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbo
Section 138. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of

Section 139. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 140. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 141. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa
Section 142. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com
Section 143. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co
Section 144. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural

Section 145. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 146. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 147. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 148. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 149. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 150. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one

Section 151. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 152. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 153. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 154. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 155. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 156. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,

Section 157. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce
Section 158. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti
Section 159. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 160. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 161. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti
Section 162. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its

Section 163. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 164. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi
Section 165. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 166. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one
Section 167. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 168. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com

Section 169. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffm
Section 170. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 171. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbo
Section 172. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patte
Section 173. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonica
Section 174. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 175. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains
Section 176. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffm
Section 177. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 178. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonica
Section 179. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com
Section 180. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi